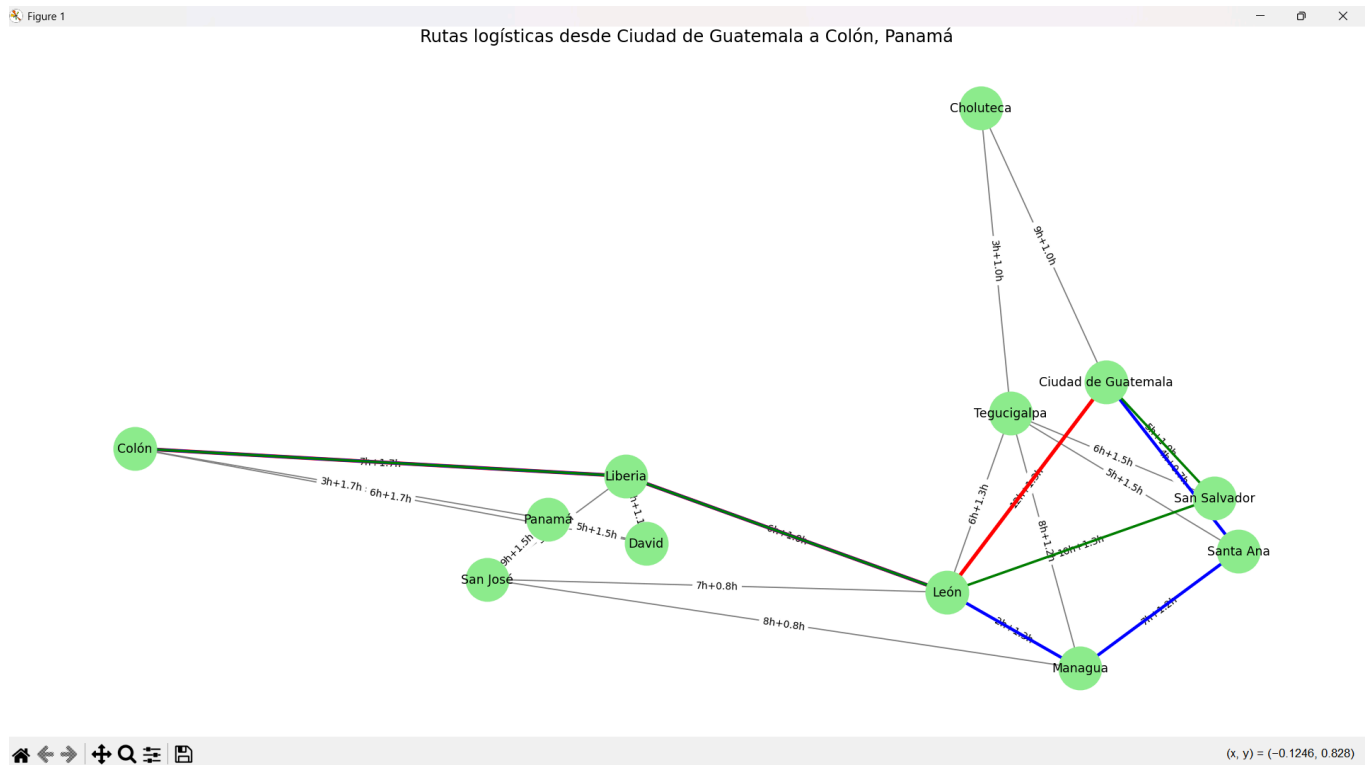


Evaluación Parcial No. 3

Ejercicio No. 1

Grafo



Resultados

Ruta más rápida encontrada

Ruta más rápida: Ciudad de Guatemala -> León -> Liberia -> Colón

- Tiempo total: 29.00 h
 - Tiempo de viaje: 25.00 h
 - Tiempo en colas de espera: 4.00 h

Otras rutas posibles

Ruta alternativa 1: Ciudad de Guatemala -> Santa Ana -> Managua -> León -> Liberia -> Colón

- Tiempo total: 31.90 h
 - Tiempo de viaje: 26.00 h

- Tiempo en colas de espera: 5.90 h

Ruta alternativa 2: Ciudad de Guatemala -> San Salvador -> León -> Liberia -> Colón

- Tiempo total: 33.00 h
 - Tiempo de viaje: 28.00 h
 - Tiempo en colas de espera: 5.00 h

Análisis

Aquí se puede ver que para determinar una "ruta más corta" no significa que es solo donde hay menos distancia de recorrido o donde menos tiempo se hace, que es lo normal que se suele llegar a pensar, pero este es un escenario más real, ya que actualmente se sabe que en la mayoría de lugares se suelen hacer colas por diferentes razones, entonces considerar estas variables para hacer el cálculo de una ruta más corta es muy importante, ya que podemos ver un escenario más real y cercano a la realidad. Entonces al contemplar las colas podemos ver que no necesariamente la ruta con menos distancia va a ser la más rápida, ya que puede ser que en distancia sea muy poco pero de colas sean unas 3 horas por ejemplo, mientras que si tomo una ruta con un poco más de distancia pero siempre avanzado puede que me haga una hora nada más.

Código

```
import networkx as nx
import matplotlib.pyplot as plt
import heapq

tiempos_espera = {
    "A": 0.0,
    "B": 1.0,
    "C": 1.5,
    "D": 1.2,
    "E": 0.8,
    "F": 1.5,
    "G": 0.7,
    "H": 1.0,
    "I": 1.3,
    "J": 1.0,
    "K": 1.1,
    "L": 1.7,
}
```

```
nombres_ciudades = {  
    "A": "Ciudad de Guatemala",  
    "B": "San Salvador",  
    "C": "Tegucigalpa",  
    "D": "Managua",  
    "E": "San José",  
    "F": "Panamá",  
    "G": "Santa Ana",  
    "H": "Choluteca",  
    "I": "León",  
    "J": "Liberia",  
    "K": "David",  
    "L": "Colón",  
}
```

```
tiempos_viaje = [  
    ("A", "B", 5),  
    ("A", "G", 4),  
    ("A", "H", 9),  
    ("A", "I", 12),  
    ("B", "C", 6),  
    ("B", "I", 10),  
    ("G", "C", 5),  
    ("G", "D", 7),  
    ("C", "D", 8),  
    ("C", "H", 3),  
    ("C", "I", 6),  
    ("D", "E", 8),  
    ("D", "I", 2),  
    ("I", "E", 7),  
    ("I", "J", 6),  
    ("E", "F", 9),  
    ("E", "J", 3),  
    ("J", "K", 4),  
    ("J", "L", 7),  
    ("K", "F", 5),  
    ("K", "L", 6),  
    ("F", "L", 3),  
]
```

```

G = nx.DiGraph()
for origen, destino, tiempo in tiempos_viaje:
    peso_total = tiempo + tiempos_espera[destino]
    G.add_edge(
        origen, destino, viaje=tiempo, espera=tiempos_espera[destino],
        peso=peso_total
    )

def dijkstra_k_rutas(G, inicio, fin, k=3):
    rutas = []
    cola = [(0, 0, inicio, [])] # (tiempo total, solo viaje, nodo actual,
    camino)
    visitadas = set()

    while cola and len(rutas) < k * 5:
        tiempo_total, viaje_total, actual, camino = heapq.heappop(cola)
        camino = camino + [actual]

        if actual == fin:
            ruta_id = tuple(camino)
            if ruta_id not in visitadas:
                rutas.append((camino, tiempo_total, viaje_total))
                visitadas.add(ruta_id)
            if len(rutas) >= k:
                break

        for vecino in G.successors(actual):
            if vecino not in camino:
                datos = G[actual][vecino]
                nuevo_viaje = viaje_total + datos["viaje"]
                nuevo_total = tiempo_total + datos["peso"]
                heapq.heappush(cola, (nuevo_total, nuevo_viaje, vecino,
                camino))

    return rutas[:k]

rutas_encontradas = dijkstra_k_rutas(G, "A", "L", k=3)

for idx, (ruta, tiempo_total, tiempo_viaje) in enumerate(rutas_encontradas):
    tipo = "Ruta más rápida" if idx == 0 else f"Ruta alternativa {idx}"

```

```

espera_total = tiempo_total - tiempo_viaje
print(f"{tipo}: {' -> '.join(nombres_ciudades[n] for n in ruta)}")
print(f"    - Tiempo total: {tiempo_total:.2f} h")
print(f"    - Tiempo de viaje: {tiempo_viaje:.2f} h")
print(f"    - Tiempo en colas de espera: {espera_total:.2f} h\n")

pos = nx.spring_layout(G, seed=42)
plt.figure(figsize=(18, 12))
nx.draw_networkx_nodes(G, pos, node_color="lightgreen", node_size=1200)
nx.draw_networkx_labels(G, pos, labels=nombres_ciudades, font_size=10)
nx.draw_networkx_edges(G, pos, edge_color="gray")
etiquetas = {(u, v): f"{G[u][v]['viaje']}h+{G[u][v]['espera']}h" for u, v in
G.edges()}

nx.draw_networkx_edge_labels(G, pos, edge_labels=etiquetas, font_size=8)
colores = ["red", "blue", "green"]
anchos = [3, 2.5, 2]

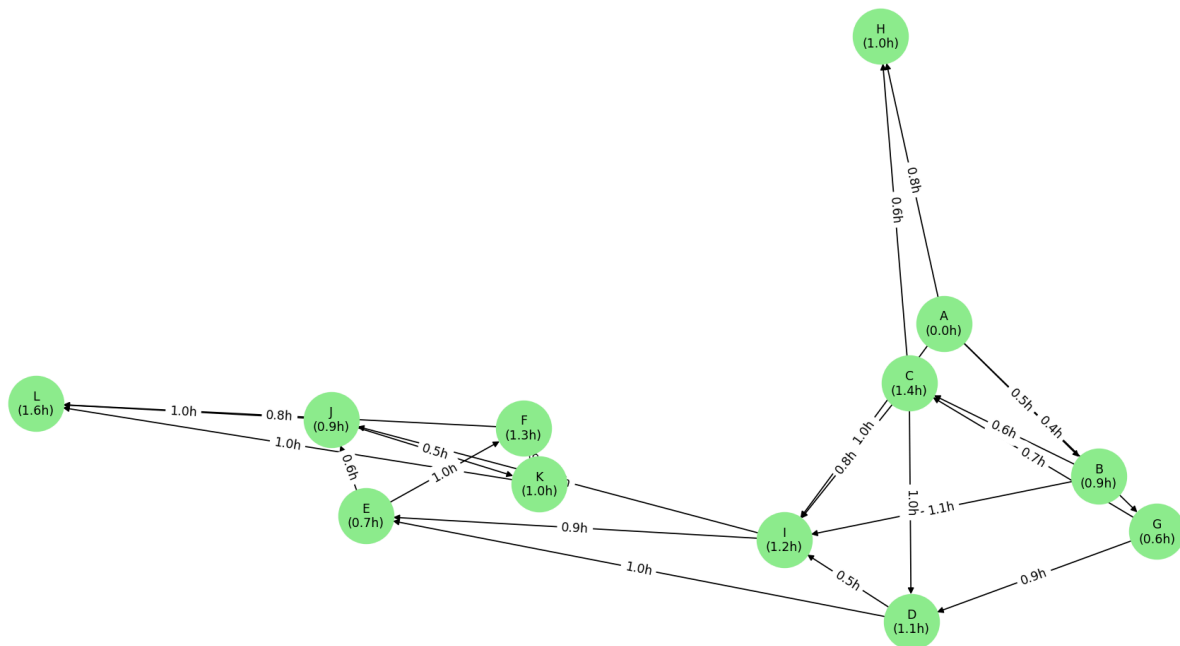
for idx, (ruta, _, _) in enumerate(rutas_encontradas):
    edges = list(zip(ruta[:-1], ruta[1:]))
    nx.draw_networkx_edges(
        G, pos, edgelist=edges, edge_color=colores[idx], width=anchos[idx]
    )

plt.title("Rutas logísticas desde Ciudad de Guatemala a Colón, Panamá",
fontsize=14)
plt.axis("off")
plt.tight_layout()
plt.show()

```

Ejercicio 2

Grafo



(x, y) = (-0.0481, 0.586)

Ruta más rápida

Ruta más rápida: A -> I -> J -> L

- Tiempo total: 6.40 h
- Tiempo de viaje: 2.70 h
- Tiempo de espera: 3.70 h

Otras rutas posibles

Ruta #2: A -> B -> I -> J -> L

- Tiempo total: 7.90 h
- Tiempo de viaje: 3.30 h
- Tiempo de espera: 4.60 h

Ruta #3: A -> I -> E -> J -> L

- Tiempo total: 7.90 h
- Tiempo de viaje: 3.50 h
- Tiempo de espera: 4.40 h

Análisis

Lo que pasa en este caso es que debido a congestiones que pueden provocar otros proveedores el tiempo total de la ruta al final se ve afectado, esto porque, aunque una ruta pueda tener un tiempo de viaje corto, si incluye universidades con largas colas de recepción, el tiempo total se incrementa. Y en este caso que se transporta comida el hecho de llegar tarde sí

es muy significativo, ya que puede ser que por llegar tarde se pierdan ventas por horas pico, es decir, si en determinada universidad el receso es donde venden más pero la comida se atrasa puede ser que esa universidad ya no venda todo porque el receso ya terminó. Y lo principal que se suele pensar es en solo costos de gasolina, cuando las pérdidas por llegar tarde también son costos que se puede decir que están ocultos y no se les presta atención.

Código

```
import heapq
import networkx as nx
import matplotlib.pyplot as plt

tiempos_espera = {
    "A": 0.0,
    "B": 0.9,
    "C": 1.4,
    "D": 1.1,
    "E": 0.7,
    "F": 1.3,
    "G": 0.6,
    "H": 1.0,
    "I": 1.2,
    "J": 0.9,
    "K": 1.0,
    "L": 1.6,
}

grafo = {
    "A": {"B": 0.5, "G": 0.4, "H": 0.8, "I": 1.0},
    "B": {"C": 0.6, "I": 1.1},
    "G": {"C": 0.7, "D": 0.9},
    "C": {"D": 1.0, "H": 0.6, "I": 0.8},
    "D": {"E": 1.0, "I": 0.5},
    "I": {"E": 0.9, "J": 0.7},
    "E": {"F": 1.0, "J": 0.6},
    "J": {"K": 0.5, "L": 1.0},
    "K": {"F": 0.9, "L": 1.0},
    "F": {"L": 0.8},
}

def dijkstra_con_rutas(grafo, tiempos_espera, inicio, destino, max_rutas=3):
```

```

heap = [(0, inicio, [])]
rutas = []

while heap and len(rutas) < max_rutas:
    tiempo_actual, nodo_actual, camino = heapq.heappop(heap)
    camino = camino + [nodo_actual]

    if nodo_actual == destino:
        rutas.append((tiempo_actual, camino))
        continue

    for vecino, tiempo_viaje in grafo.get(nodo_actual, {}).items():
        if vecino not in camino:
            tiempo_total = tiempo_actual + tiempo_viaje +
tiempos_espera[vecino]
            heapq.heappush(heap, (tiempo_total, vecino, camino))
return rutas

def desglose_tiempos(camino):
    tiempo_viaje = 0
    tiempo_espera_total = 0
    for i in range(len(camino) - 1):
        tiempo_viaje += grafo[camino[i]][camino[i + 1]]
    for nodo in camino[1:]: # no contar espera en nodo inicial
        tiempo_espera_total += tiempos_espera[nodo]
    return tiempo_viaje, tiempo_espera_total

# obtener rutas
rutas = dijkstra_con_rutas(grafo, tiempos_espera, "A", "L", max_rutas=10)
rutas_unicas = []
usadas = set()

for tiempo_total, ruta in rutas:
    ruta_str = "->".join(ruta)
    if ruta_str not in usadas:
        rutas_unicas.append((tiempo_total, ruta))
        usadas.add(ruta_str)
    if len(rutas_unicas) == 3:
        break

```

```

for idx, (tiempo_total, ruta) in enumerate(rutas_unicas):
    viaje, espera = desglose_tiempos(ruta)
    print(f"\nRuta #{idx+1}: {' -> '.join(ruta)}")
    print(f" - Tiempo total: {tiempo_total:.2f} h")
    print(f" - Tiempo de viaje: {viaje:.2f} h")
    print(f" - Tiempo de espera: {espera:.2f} h")

G = nx.DiGraph()
for origen, destinos in grafo.items():
    for destino, peso in destinos.items():
        G.add_edge(origen, destino, weight=peso)

pos = nx.spring_layout(G, seed=42)
labels = {nodo: f"{nodo}\n({tiempos_espera[nodo]}h)" for nodo in G.nodes}
plt.figure(figsize=(12, 8))
nx.draw(
    G,
    pos,
    with_labels=True,
    labels=labels,
    node_size=2000,
    node_color="lightgreen",
    font_size=10,
)
nx.draw_networkx_edge_labels(
    G, pos, edge_labels={(u, v): f"{d['weight']}h" for u, v, d in
G.edges(data=True)}}
)
plt.title("Red de distribución de UniCafé entre universidades")
plt.show()

```